

Java Backend Architecture

Interview Series

Chapter 12.4

Date & Time Localization

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 12.4 – Date & Time Localization

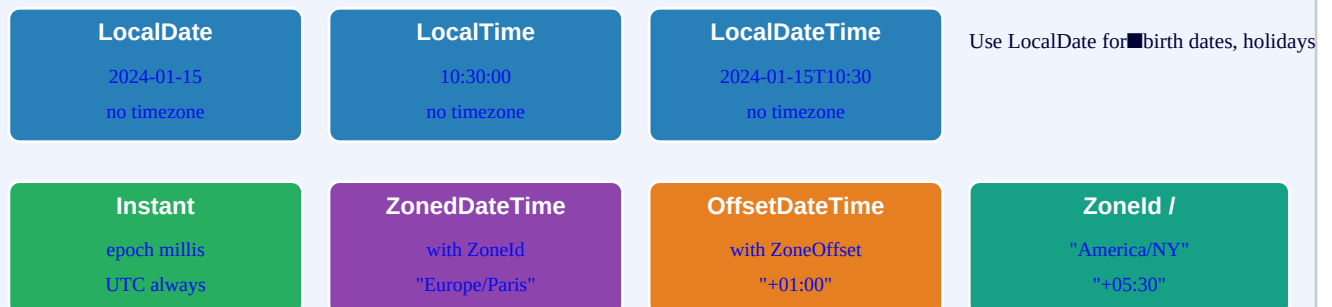
Introduction

Date and time localisation is among the trickiest areas in i18n because it combines three orthogonal concerns: the calendar system (Gregorian, Japanese, Buddhist, Hijri), the timezone (UTC, America/New_York, Asia/Tokyo), and the display format (MM/dd/yyyy in the US vs dd.MM.yyyy in Germany). Java's `java.time` package (introduced in Java 8, based on Joda-Time) cleanly separates these: `LocalDate/LocalTime/LocalDateTime` carry no timezone information and are suitable for calendar-like data (birthdays, holidays); `Instant` is a UTC epoch timestamp; `ZonedDateTime` combines a datetime with a `ZoneId`.

For API design, ISO 8601 is the unambiguous standard: `"2024-01-15T10:30:00Z"` (UTC) or `"2024-01-15T10:30:00+05:30"` (with offset). Never use locale-formatted dates in API responses — `"01/15/24"` is ambiguous (is it January 15 or the 15th day of month 01 in a non-US locale?). For UI display, `DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT).withLocale(locale)` produces the locale-correct representation. Jackson's `@JsonFormat` and Spring's `@DateTimeFormat` bridge between API date strings and Java types.

java.time Class Hierarchy

java.time Class Hierarchy and Use Cases



ISO 8601 in APIs: `2024-01-15T10:30:00Z` (`Instant/UTC`) or `2024-01-15T10:30:00+01:00` (`OffsetDateTime`)

DateTimeFormatter: ISO vs Locale-Aware

DateTimeFormatter: ISO vs Locale-Aware Styles

Formatter	en_US Result	fr_FR Result	Notes
ISO_LOCAL_DATE	2024-01-15	2024-01-15	Both same — ISO is locale-independent
ofLocalizedDate(SHORT)	1/15/24	15/01/2024	Locale-aware short date
ofLocalizedDate(MEDIUM)	Jan 15, 2024	15 janv. 2024	Locale-aware medium date
ofLocalizedDate(LONG)	January 15, 2024	15 janvier 2024	Locale-aware long date
ofPattern("dd/MM/yyyy")	15/01/2024	15/01/2024	Fixed pattern (NOT locale-aware)

Use ISO_LOCAL_DATE for APIs; ofLocalizedDate() for UI; avoid ofPattern() for locale-sensitive display.

Calendar Systems

Calendar Systems for the Same Day (ISO: 2024-01-15)

Gregorian / ISO	2024-01-15	International standard; basis of ISO 8601
Japanese	Reiwa 6-01-15	Emperor era-based; required for Japanese govt apps
Buddhist	2567-01-15	Used in Thailand (+543 Gregorian years)
Hijri (Islamic)	1445-07-04	Lunar calendar; date shifts each year
Hebrew	5784-11-05	Used in Israel; Tishri-based year

Java: JapaneseChronology, ThaiBuddhistChronology, HijrahChronology, IsoChronology (default)

Key Definitions

LocalDate	Date without time or timezone (2024-01-15); use for birthdays, holidays, due dates
LocalTime	Time without date or timezone (10:30:00); use for business hours, recurring times
LocalDateTime	Date + time without timezone; use only when timezone is implicit/fixed in context
Instant	UTC epoch moment (nanosecond precision); use for event timestamps, DB storage, API exchange
ZonedDateTime	Date + time + ZoneId (full DST rules); use for user-facing scheduling with DST awareness
OffsetDateTime	Date + time + fixed ZoneOffset (+01:00); no DST rules; preferred in APIs over ZonedDateTime
ZoneId	IANA timezone database ID: "America/New_York", "Europe/Paris"; includes DST rules
DateTimeFormatter	Thread-safe formatter/parser; ISO_LOCAL_DATE for APIs, ofLocalizedDate(style) for UI

FormatStyle	SHORT, MEDIUM, LONG, FULL — locale-aware date/time style constants for ofLocalizedDate/Time
ISO 8601	International standard for dates: yyyy-MM-dd, times: HH:mm:ss, combined: yyyy-MM-ddTHH:mm:ssZ
Period	Date-based duration: P1Y2M3D (1 year, 2 months, 3 days); use with LocalDate arithmetic
Duration	Time-based duration: PT4H30M (4 hours, 30 min); use with Instant/LocalTime arithmetic
@JsonFormat	Jackson annotation specifying date format and timezone for serialization: @JsonFormat(pattern="yyyy-MM-dd")
@DateTimeFormat	Spring annotation for binding date strings in form data: @DateTimeFormat(iso=ISO.DATE)
Relative time	Display "2 hours ago", "yesterday" using ChronoUnit.between() and locale-aware strings

java.time Types: When to Use Which

Type	Has Timezone	Use Case	API/DB Format
LocalDate	No	Birthdays, holidays, due dates	yyyy-MM-dd
LocalTime	No	Business hours, recurring times	HH:mm:ss
LocalDateTime	No	Timestamps where TZ is implicit	yyyy-MM-ddTHH:mm:ss
Instant	UTC	Event timestamps, audit logs	yyyy-MM-ddTHH:mm:ssZ
OffsetDateTime	Fixed offset	API responses with offset	yyyy-MM-ddTHH:mm:ss+HH:mm
ZonedDateTime	IANA zone	User calendar, DST-aware scheduling	ISO 8601 with zone

Code Examples

1. Creating and formatting dates with java.time

```
import java.time.*; import java.time.format.*; import java.util.Locale; // Create dates
LocalDate date = LocalDate.of(2024, 1, 15); LocalTime time = LocalTime.of(10, 30, 0);
LocalDateTime ldt = LocalDateTime.of(date, time); Instant now = Instant.now(); ZonedDateTime
zdt = ZonedDateTime.of(ldt, ZoneId.of("America/New_York")); // ISO formatting (for APIs)
System.out.println(date.format(DateTimeFormatter.ISO_LOCAL_DATE)); // 2024-01-15
System.out.println(zdt.format(DateTimeFormatter.ISO_OFFSET_DATE_TIME)); //
2024-01-15T10:30:00-05:00 System.out.println(now.toString()); // 2024-01-15T15:30:00Z //
Locale-aware display formatting DateTimeFormatter shortFmt = DateTimeFormatter
.ofLocalizedDate(FormatStyle.SHORT) .withLocale(Locale.US);
System.out.println(date.format(shortFmt)); // 1/15/24 DateTimeFormatter frFmt =
DateTimeFormatter .ofLocalizedDate(FormatStyle.LONG) .withLocale(Locale.FRANCE);
System.out.println(date.format(frFmt)); // 15 janvier 2024 // Parsing LocalDate parsed =
LocalDate.parse("2024-01-15", DateTimeFormatter.ISO_LOCAL_DATE); LocalDate parsedFr =
LocalDate.parse("15/01/2024", DateTimeFormatter.ofPattern("dd/MM/yyyy"));
```

2. Jackson date serialization configuration

```
// application.properties spring.jackson.serialization.write-dates-as-timestamps=false
spring.jackson.date-format=yyyy-MM-dd'T'HH:mm:ss'Z' spring.jackson.time-zone=UTC //
ObjectMapper configuration bean @Bean public ObjectMapper objectMapper() { ObjectMapper mapper
= new ObjectMapper(); mapper.registerModule(new JavaTimeModule());
mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
mapper.setTimeZone(TimeZone.getTimeZone("UTC")); return mapper; } // DTO with annotations
public class EventDto { // ISO date only @JsonFormat(pattern = "yyyy-MM-dd") private LocalDate
eventDate; // Full ISO 8601 with offset @JsonFormat(pattern = "yyyy-MM-dd'T'HH:mm:ssXXX",
timezone = "UTC") private OffsetDateTime createdAt; // Input parsing @JsonDeserialize(using =
LocalDateDeserializer.class) @JsonSerialize(using = LocalDateSerializer.class) private
LocalDate dueDate; } // Output:
{"eventDate":"2024-01-15","createdAt":"2024-01-15T10:30:00+00:00"}
```

3. Spring @DateTimeFormat and form binding

```
// Spring MVC form binding public class BookingForm { @DateTimeFormat(iso =
DateTimeFormat.ISO.DATE) private LocalDate checkIn; // accepts "2024-01-15"
@DateTimeFormat(pattern = "dd/MM/yyyy") private LocalDate checkOut; // accepts "15/01/2024"
@DateTimeFormat(iso = DateTimeFormat.ISO.DATE_TIME) private LocalDateTime appointmentAt; //
"2024-01-15T10:30:00" } // REST controller – request param binding @GetMapping("/bookings")
public List<Booking> getBookings( @RequestParam @DateTimeFormat(iso = ISO.DATE) LocalDate
from, @RequestParam @DateTimeFormat(iso = ISO.DATE) LocalDate to) { return
bookingService.findBetween(from, to); } // GET /bookings?from=2024-01-15&to=2024-01-31 // For
query parameter with timezone: @GetMapping("/events") public List<Event> getEvents(
@RequestParam @DateTimeFormat(iso = ISO.DATE_TIME) ZonedDateTime after) { return
eventService.findAfter(after); } // GET /events?after=2024-01-15T10:30:00-05:00
```

4. Date arithmetic with Period and Duration

```
// Period: date-based (years, months, days) LocalDate start = LocalDate.of(2024, 1, 15);
LocalDate end = LocalDate.of(2024, 4, 20); Period period = Period.between(start, end);
System.out.println(period); // P3M5D (3 months 5 days) System.out.println(period.getMonths());
// 3 System.out.println(period.getDays()); // 5 LocalDate nextMonth =
start.plus(Period.ofMonths(1)); // 2024-02-15 LocalDate nextYear = start.plusYears(1); //
2025-01-15 // Duration: time-based (hours, minutes, seconds, nanos) Instant t1 =
Instant.parse("2024-01-15T10:00:00Z"); Instant t2 = Instant.parse("2024-01-15T14:30:00Z");
Duration duration = Duration.between(t1, t2); System.out.println(duration); // PT4H30M
System.out.println(duration.toHours()); // 4 System.out.println(duration.toMinutes()); // 270
// ChronoUnit for simple differences long days = ChronoUnit.DAYS.between(start, end); // 96
long hours = ChronoUnit.HOURS.between(t1, t2); // 4 // Relative time display ("2 hours ago")
Duration ago = Duration.between(eventTime, Instant.now()); String relative; if
(ago.toMinutes() < 1) relative = "just now"; else if (ago.toHours() < 1) relative =
ago.toMinutes() + " minutes ago"; else if (ago.toDays() < 1) relative = ago.toHours() + " hours
ago"; else relative = ago.toDays() + " days ago";
```

5. Locale-aware first-day-of-week and calendar week

```
import java.time.temporal.WeekFields; // First day of week is locale-dependent WeekFields
usWeek = WeekFields.of(Locale.US); // SUNDAY WeekFields euWeek = WeekFields.of(Locale.FRANCE);
// MONDAY LocalDate date = LocalDate.of(2024, 1, 15); // Monday // US week number (Sunday
start) int usWeekNum = date.get(usWeek.weekOfWeekBasedYear()); // week 3 // EU week number
(Monday start, ISO 8601) int euWeekNum = date.get(euWeek.weekOfWeekBasedYear()); // week 3
(Monday = start) // ISO 8601 week number (always Monday start) int isoWeek =
date.get(WeekFields.ISO.weekOfWeekBasedYear()); // 3 int isoDow =
```

```
date.get(WeekFields.ISO.dayOfWeek()); // 1 (Monday=1 in ISO) // First day of the week for a
given locale DayOfWeek firstDay = WeekFields.of(Locale.US).getFirstDayOfWeek(); // SUNDAY
DayOfWeek firstDayEu = WeekFields.of(Locale.GERMANY).getFirstDayOfWeek(); // MONDAY //
Building a week-based calendar grid requires knowing the first day: LocalDate startOfWeek =
date.with(TemporalAdjusters.previousOrSame(firstDay)); // US: 2024-01-14 (Sunday), EU:
2024-01-15 (Monday already)
```

6. Database storage of dates and Jackson integration

```
// JPA entity with proper date types @Entity public class Appointment { @Id @GeneratedValue
private Long id; // Store as DATE column (no time, no timezone) @Column(name =
"appointment_date") private LocalDate appointmentDate; // maps to SQL DATE // Store as
TIMESTAMP WITH TIME ZONE in PostgreSQL @Column(name = "created_at", columnDefinition =
"TIMESTAMPTZ") private OffsetDateTime createdAt; // maps to SQL TIMESTAMP WITH TIMEZONE //
Avoid: java.util.Date, Calendar (legacy, mutable, messy) // Avoid: LocalDateTime for timestamps
(loses timezone info) } // application.properties for Hibernate: //
spring.jpa.properties.hibernate.jdbc.time_zone=UTC // Flyway migration: // CREATE TABLE
appointment ( // id BIGSERIAL PRIMARY KEY, // appointment_date DATE NOT NULL, -- LocalDate //
created_at TIMESTAMPTZ NOT NULL -- OffsetDateTime / Instant // ); // Jackson serialization
test: @Test void appointmentSerializesCorrectly() throws Exception { Appointment a = new
Appointment(); a.setAppointmentDate(LocalDate.of(2024, 1, 15));
a.setCreatedAt(OffsetDateTime.parse("2024-01-15T10:30:00+00:00")); String json =
objectMapper.writeValueAsString(a); assertThat(json).contains("2024-01-15");
assertThat(json).contains("2024-01-15T10:30:00Z"); }
```

Interview Questions & Answers

Q1. When should you use LocalDate vs ZonedDateTime vs Instant?

LocalDate: for calendar dates without time meaning — birthdays, holidays, contract due dates. There is no 'timezone' for a birthday — January 15 means January 15 everywhere. **ZonedDateTime:** for user-facing scheduled events that must account for DST — a meeting at 9am New York that should still be 9am after DST transition. **Instant:** for point-in-time event records — audit logs, message timestamps, Kafka offsets. **Instant** has no timezone; it's a UTC epoch moment. **OffsetDateTime:** like **ZonedDateTime** but with a fixed offset — preferred in APIs because it's unambiguous and has no DST rules.

Q2. What is ISO 8601 and why should it be used in REST APIs?

ISO 8601 is the international standard for date/time representation: dates as '2024-01-15', times as '10:30:00', combined as '2024-01-15T10:30:00Z' (Z = UTC) or '2024-01-15T10:30:00+05:30' (with offset). It is: (1) unambiguous — '01/02/03' is ambiguous (Jan 2? Feb 1? Feb 3?); ISO 8601 is not. (2) Sortable — lexicographic order = chronological order. (3) Universal — parseable by all modern frameworks in all languages. Always use ISO 8601 in REST APIs; never use locale-formatted dates like 'January 15, 2024' in JSON responses.

Q3. Explain the difference between OffsetDateTime and ZonedDateTime.

OffsetDateTime has a fixed UTC offset (+01:00, -05:00): '2024-01-15T10:30:00+01:00'. The offset doesn't change with DST. **ZonedDateTime** has a named timezone (ZonedId) like 'Europe/Paris', which includes full DST rules: in summer the offset is +02:00, in winter +01:00. If you store a **ZonedDateTime** and the timezone rules change (governments change DST rules), the stored offset may become invalid. For APIs: prefer **OffsetDateTime** — it's explicit and has no hidden DST complexity. For scheduling future recurring events: use **ZonedId** + **LocalDateTime** and compute the next occurrence using current timezone rules.

Q4. How does `DateTimeFormatter.ofLocalizedDate` work and when should you use it?

`ofLocalizedDate(FormatStyle)` creates a formatter that uses the locale's culturally appropriate format for the given style. SHORT: '1/15/24' (en_US), '15/01/24' (fr_FR). MEDIUM: 'Jan 15, 2024' (en_US), '15 janv. 2024' (fr_FR). LONG: 'January 15, 2024' (en_US), '15 janvier 2024' (fr_FR). Chain `.withLocale(userLocale)` to set the locale. Use for UI display only. For API responses, always use `ISO_LOCAL_DATE` or `ISO_OFFSET_DATE_TIME`. `DateTimeFormatter` is thread-safe and can be stored as a static constant (unlike the legacy `SimpleDateFormat` which is not thread-safe).

Q5. Why is `SimpleDateFormat` not thread-safe and what is the modern replacement?

`SimpleDateFormat` holds internal mutable parse state (a `Calendar` object). Concurrent access from multiple threads corrupts this state, causing incorrect results or exceptions. Common bugs: `format()` returning wrong month, `parse()` throwing unpredictably under load. Fixes: (1) Create a new `SimpleDateFormat` per use (safe but expensive). (2) Use `ThreadLocal`. (3) Synchronize access. Modern fix: `DateTimeFormatter` (java.time) is immutable and thread-safe — store as a static final constant and share freely. `DateTimeFormatter` also has a richer API and better ISO 8601 support.

Q6. How do you handle date serialisation in Spring Boot with Jackson?

Add `jackson-datatype-jsr310` (included in `spring-boot-starter-web`) and register `JavaTimeModule`. Set `spring.jackson.serialization.write-dates-as-timestamps=false` to emit ISO 8601 strings instead of epoch arrays ([2024,1,15]). For specific formats: `@JsonFormat(pattern='yyyy-MM-dd')` on a `LocalDate` field. For timezone: set `spring.jackson.time-zone=UTC` in properties or `@JsonFormat(timezone='UTC')`. Never use `java.util.Date` in DTOs — use `LocalDate`, `LocalDateTime`, `OffsetDateTime`, or `Instant`. Deserialisation works symmetrically: Jackson parses the ISO string back to the Java type.

Q7. Explain Period vs Duration and when to use each.

`Period` is date-based (years, months, days) and respects calendar rules: `Period.ofMonths(1)` added to Jan 31 gives Feb 28 (last day of February), not Feb 31. Use for human-meaningful durations: subscription periods, loan tenors, age. `Duration` is time-based (hours, minutes, seconds, nanoseconds) — fixed 24-hour days: `Duration.ofDays(1) = 86400` seconds, even across DST transitions. Use for timeouts, SLAs, rate limiting windows. `ChronoUnit.DAYS.between(d1, d2)` counts calendar days (respects DST in `ZonedDateTime`). Never use `Period` where `Duration` is needed or vice versa.

Q8. How do you display relative time ('2 hours ago') in a Java application?

Compute the `Duration` between the event time and now: `Duration.between(eventInstant, Instant.now())`. Then apply display rules: < 60 seconds → 'just now', < 60 minutes → N minutes ago, < 24 hours → N hours ago, < 7 days → N days ago, else show the full date. Localise the units using `ResourceBundle`: `'time.minutes.ago={0} minutes ago'`. For production, use a library: `timeago4j`, `PrettyTime`, or `ICU4J RelativeDateTimeFormatter` which handles plural forms correctly per locale (e.g., '2 minutes ago' vs '2 Minuten zuvor' in German). Android also provides `DateUtils.getRelativeTimeSpanString`.

Q9. How do you store dates in a PostgreSQL database from Spring/JPA?

Use `TIMESTAMPTZ` (`TIMESTAMP WITH TIME ZONE`) for timestamps — PostgreSQL normalises to UTC on storage and converts on display per session timezone. Map to `OffsetDateTime` or `Instant` in Java. For dates without time, use `DATE` — map to `LocalDate`. Set `spring.jpa.properties.hibernate.jdbc.time_zone=UTC` so Hibernate sends/receives values in UTC. Without this, Hibernate uses the JVM timezone, causing silent offset errors. In JDBC: use `PreparedStatement.setObject(n, localDate)` with java.time types directly (JDBC 4.2+). Never use `setDate()` with `java.sql.Date` for timestamps — it loses time.

Q10. What are the Japanese and Buddhist calendar systems and how does Java handle them?

Japan uses imperial era years: Reiwa 6 (2024), Heisei 31 (2019), Showa 64 (1989). Java's `JapaneseChronology` and `JapaneseDate` support this: `JapaneseDate.of(JapaneseEra.REIWA, 6, 1, 15)`. Thailand uses the Buddhist calendar (+543 years): 2024 = 2567 BE. `ThaiBuddhistChronology` handles this. `DateTimeFormatter.ofLocalizedDate(FULL).withChronology(JapaneseChronology.INSTANCE).withLocale(Locale.JAPAN)` produces era-formatted dates. In practice: store internally as ISO dates (`LocalDate`), convert to the regional calendar for display only using the `ChronoLocalDate` API.

Q11. Why is first-day-of-week locale-dependent and how do you handle it in a calendar UI?

The US (`en_US`) starts weeks on Sunday; most of Europe, ISO 8601, and most of the world start on Monday; some Middle Eastern countries start on Saturday. `WeekFields.of(locale).getFirstDayOfWeek()` returns the correct starting day. For calendar grid rendering: compute `startOfWeek = date.with(TemporalAdjusters.previousOrSame(firstDay))`. This affects: week number calculation (US week 1 vs ISO week 1 differ), calendar column headers, and date range queries ('this week'). Always use `WeekFields` — never hardcode `DayOfWeek.SUNDAY` as the week start.

Q12. How do you handle the 'spring forward / fall back' edge case in date arithmetic?

When clocks spring forward (e.g., 2am → 3am, so 2:30am doesn't exist) or fall back (2am → 1am, so 1:30am happens twice), `ZonedDateTime` arithmetic can produce surprises. `Instant + Duration` gives the correct elapsed time (ignores DST). `ZonedDateTime.plusHours(1)` at the DST gap skips the missing hour correctly. For scheduling: store the `ZonedDateTime` and compute the next wall-clock time on demand — this handles future rule changes. A meeting at 9am New York recurs at 9am regardless of DST. An `Instant`-based reminder fires at the same UTC moment regardless of DST — different semantics.

Q13. How does Jackson handle `Instant` vs `LocalDateTime` serialisation by default?

With `JavaTimeModule` and `write-dates-as-timestamps=true` (default): `Instant` → [seconds, nanos] array; `LocalDateTime` → [2024,1,15,10,30,0] array — ugly and not interoperable. Set `write-dates-as-timestamps=false`: `Instant` → '2024-01-15T10:30:00Z'; `LocalDateTime` → '2024-01-15T10:30:00'. Note: `LocalDateTime` has no timezone in output — if the client is in a different timezone, it may misinterpret the value. Prefer `OffsetDateTime` for API fields that represent a point in time, not a 'wall clock' time.

Q14. What is the difference between ISO week year and calendar year?

ISO 8601 defines a week-based year: week 1 is the week containing the first Thursday of the year (equivalently, the week with January 4). This means early January days may belong to the previous ISO year (e.g., Jan 1 2021 (Friday) is in ISO week 53 of 2020), and late December days may belong to the next ISO year. Java: `date.get(WeekFields.ISO.weekBasedYear())` gives the ISO week year. Using 'YYYY' in `DateTimeFormatter` pattern gives the ISO week year; 'yyyy' gives the calendar year. Common bug: 'yyyy-MM-dd' vs 'YYYY-MM-dd' — the latter is almost always wrong.

Q15. How would you implement a 'booking date range' query that works correctly across timezones?

The user selects 'January 15 to January 20' in their local timezone (e.g., Tokyo +09:00). In UTC, this is Jan 14 15:00Z to Jan 19 15:00Z. Strategy: (1) Accept dates as `LocalDate` (no timezone) — this represents the user's calendar intent. (2) Convert to a range: `startInstant = localDateStart.atStartOfDay(userZoneId).toInstant()`; `endInstant = localDateEnd.plusDays(1).atStartOfDay(userZoneId).toInstant()`. (3) Query DB: `WHERE booking_time >= ? AND booking_time < ?` with UTC instants. Never pass a `LocalDate` directly to a timestamp comparison — you'll miss timezone-offset bookings.

Q16. How do you prevent date parsing bugs when consuming a third-party API?

Steps: (1) Check the API documentation for the date format (ISO 8601, epoch millis, proprietary). (2) Use explicit `DateTimeFormatter` patterns: `DateTimeFormatter.ofPattern('EEE, dd MMM yyyy HH:mm:ss z', Locale.ENGLISH)` for RFC 2822 (email/RSS dates). (3) Handle timezone: if the API returns dates in a specific timezone, parse as `OffsetDateTime` or `ZonedDateTime` and convert to UTC immediately. (4) Write contract tests: use `WireMock` to simulate the API response with known dates and assert the parsed value. (5) Add monitoring: alert on `ParseException` in production — APIs sometimes change date formats without notice.

Q17. Explain Clock injection for testable date/time code in Java.

Inject `java.time.Clock` into services instead of calling `Instant.now()` or `LocalDate.now()` directly. In production: bind `Clock.systemUTC()` in the Spring context. In tests: `Clock.fixed(Instant.parse('2024-01-15T10:00:00Z'), ZoneOffset.UTC)` gives a deterministic clock. Service: `this.now = clock.instant(); LocalDate today = LocalDate.now(clock)`. Tests can set the clock to any point in time to test expiry logic, scheduling, and time-sensitive business rules. This is the correct alternative to mocking static methods like `Instant.now()` with `Mockito`.

Q18. How do you format a date for display in a Thymeleaf template using the user's locale?

Use Thymeleaf's `#temporals` utility: `${#temporals.format(event.date, 'dd/MM/yyyy')}` for a fixed pattern, or `${#temporals.formatISO(event.date)}` for ISO format. For locale-aware formatting, use Thymeleaf's `format` with the locale implicitly applied via Spring's `LocaleContextHolder`: `${#temporals.format(event.date, 'dd MMM yyyy', #locale)}`. Alternatively, format the date in the controller before passing to the model using `DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM).withLocale(locale)` and pass the formatted string. The controller approach gives more control and is easier to test.

Q19. What happens when you compare two `LocalDateTime` values from different timezones?

`LocalDateTime` has no timezone — comparison is purely by calendar value. `LocalDateTime.of(2024,1,15,10,0)` compared to itself from two different timezones appears equal even though they represent different instants (Tokyo 10am $\neq$ London 10am). This is the danger of using `LocalDateTime` for timestamps: it silently loses timezone information. Fix: always convert to `Instant` before comparison: `ldt.atZone(ZoneId.of(userTimezone)).toInstant()`. Or use `OffsetDateTime/ZonedDateTime` throughout. Never compare `LocalDateTimes` that originated from different timezones.

Q20. Describe how to implement an internationalised event reminder system that sends emails at correct local times.

Architecture: (1) Store the event as `LocalDateTime` + `ZoneId` in the DB (e.g., `2024-03-15T09:00 + 'America/New_York'`). Do NOT store as a UTC timestamp — if DST rules change between scheduling and the event date, the wall clock time should stay at 9am. (2) A scheduler job runs every minute: query events where `nextReminderAt <= now()`. Compute `nextReminderAt = localDateTime.minusMinutes(30).atZone(zoneId).toInstant()`. (3) On send, format the date for the user's locale: `DateTimeFormatter.ofLocalizedDateTime(MEDIUM).withLocale(userLocale).withZone(userZoneId).format(eventInstant)`. (4) Recompute `nextReminderAt` after each DST transition using the latest IANA timezone database.